

GUÍA RESUMEN — Debugging Flutter

Resumen integral del debugging y la optimización en Flutter sin volver a leer el módulo completo. Hilo conductor: Modos de ejecución → launch.json y breakpoints avanzados → Debug Console → DevTools (Inspector, Performance, Memory, CPU, Network, Logging, App Size) → Rendimiento (rebuilds, memory leaks, rendering) → Debugging programático → Asíncrono → Workflow por tipo de bug → Mentalidad anti-IA → Playbook sistemático → Ejecución aislada por capa → Prácticas, cheatsheets y métricas.

0 Modos de ejecución y fundamentos

DEBUG · PROFILE · RELEASE · HOT RELOAD

Todo el módulo arranca de elegir el modo correcto: el debugger y DevTools viven en `debug` / `profile` ; la **medición de rendimiento solo vale en `profile`** (en debug el JIT no optimiza, 10-100× más lento y con asserts activados). Un frame pasa por **UI thread** (build → layout → paint, código Dart/BLoC) y **Raster thread** (pintado GPU; la compilación de shaders es fuente típica de jank).

Modo	Para qué	Optimizaciones
debug	Depurar con breakpoints, hot reload	Ninguna (lento)
profile	Medir rendimiento (siempre aquí)	Mayoría habilitadas
release	Publicar, verificar builds	Todas

Hot Reload vs Hot Restart

	Hot Reload (<code>r</code>)	Hot Restart (<code>R</code> / <code>Ctrl+Shift+F5</code>)
Estado	Conserva (BLoC, scroll, controllers, breakpoints)	Reinicia todo (vuelve a <code>initState()</code>), limpia breakpoints
Cuándo	Cambios visuales en <code>build()</code>	<code>initState()</code> , <code>main()</code> , <code>const</code> , imports, static fields, config
Duración	~1-2 s	~5-10 s

Regla de oro: para debugging usa `debug` ; para medir performance usa `profile` — nunca midas en debug (el JIT distorsiona todo). Flag clave: `--dart-define` para config por entorno y `--start-paused` para esperar al debugger.

`launch.json` (.vscode/launch.json) define configuraciones reutilizables: comando, plataforma, args y attach. Con `"request": "attach"` el debugger se conecta a una app ya corriendo sin copiar la URI (solo en `debug` / `profile` ; release no expone VM Service).

Clave	Uso
<code>request</code>	<code>launch</code> <code>attach</code>
<code>program</code>	<code>lib/main.dart</code> (o <code>test/x_test.dart</code>)
<code>flutterMode</code>	<code>debug</code> · <code>profile</code> · <code>release</code> (preferir a <code>toolArgs</code>)
<code>dartDefine</code>	<code>["KEY=value"]</code> → <code>--dart-define</code> (config por entorno)
<code>toolArgs</code>	Args de <code>flutter run</code> : <code>-d device</code> , <code>--flavor</code> , <code>-t entrypoint</code>
<code>args</code>	Args del programa/test, ej. <code>--name "login exitoso"</code>
<code>compounds</code>	Lanzar varias configuraciones a la vez (app + worker isolate)

Los 8 tipos de breakpoint

Tipo	Pausa	Uso
Normal	Sí	Pausa en una línea (<code>F9</code>)
Condicional	Sí	Solo si la condición es <code>true</code> : <code>state is LoginError</code> , <code>items.length > 100</code>
Hit Count	Sí	Tras N hit: <code>== 1</code> , <code>> 10</code> (ej. frame 30 de una animación)
Triggered	Sí	Se dispara cuando otro breakpoint se activó antes
Inline	Sí	Columna exacta de una línea (<code>Ctrl+Shift+F9</code> en sesión)
Function	Sí	Por nombre: <code>setState</code> , <code>Navigator.push</code> (fallback: línea)
Data	Soporte limitado	Cuando el valor de una variable cambia
Logpoint	No	<code>"Email: {event.email}"</code> logea sin pausar (diamante azul)

Regla: usa logpoints en lugar de `print()` para debugging temporal — no contaminan el código, pueden ser condicionales y no se commitean. Seguridad: nunca pongas anon keys/secretos en `launch.json` ; usa `String.fromEnvironment` desde `.env` o variables del sistema.

2 Inspección de datos y Debug Console

- **VARIABLES:** valores locales durante el pause. **Copy as Expression** (pega en WATCH) y **Set Value** te permiten simular edge cases (ej. `event.email = ""`) sin tocar código.
- **WATCH:** expresiones Dart reevaluadas en cada pause.
- **Debug Console = REPL** (`Ctrl+Shift+Y`): ejecuta código en el contexto de la sesión — crear objetos, `await` , modificar variables, multiline con `Shift+Enter` .
- **Multi-target:** config por device (android, ios, chrome) y debugging remoto por red (`adb connect` , `--web-port`).

```
// Expresiones útiles para WATCH
state.runtimeType
state.items.length
user != null
users.map((u) => u.name).toList()
streamController.hasListener
DateTime.now().difference(startTime).inSeconds

// Debug Console como laboratorio (Sin tocar código)
bloc.state // → LoginLoading
await loginUseCase(LoginParams(email: "t@t.com", password: "12345678"))
state is LoginLoading ? "cargando" : "listo"
```

Flujo típico: breakpoint → inspeccionar VARIABLES → evaluar en Debug Console → avanzar con `F10` / `F11` (Step Over/Into) → `F5` Continue. Prueba la lógica en el REPL antes de escribir unit tests.

DevTools es la suite de profiling y diagnóstico de Flutter (no un debugger). Conecta con la app vía **VM Service** (socket sobre DDS). Cómo abrirla: pulsa `d` en el terminal de `flutter run`, o `Ctrl+Shift+P → Dart: Open DevTools`, o `dart devtools <vm-service-uri>`. **Solo funciona en `--debug` / `--profile`, nunca en release.**

Vista	Para qué
Flutter Inspector	Widget tree, layout, constraints
Performance	Frames, jank, flame chart
Memory	Heap, leaks, GC, snapshots
CPU Profiler	Flame chart por sampling
Network	Requests HTTP y WebSocket
Debugger	Breakpoints sincronizados con VS Code
Logging	Logs de <code>package:logging</code> y print
App Size	Análisis del tamaño del build

Limitaciones: Network solo ve requests hechos *desde Dart* (los nativos quedan invisibles); los snapshots de memoria son aproximados en debug; debug mode añade overhead de rendimiento. El Debugger de DevTools comparte VM Service con VS Code: los breakpoints se sincronizan solos.

Flutter Inspector: Widget Tree / Details / **Layout Explorer** (distingue constraints impuestas por el padre vs tamaño real, p.ej. `0 ≤ w ≤ 411, 0 ≤ h ≤ ∞`). **Select Widget Mode** localiza en la app el widget que se ve mal. MediaQuery responsive: `MediaQuery.sizeOf(context)`.

Debug Paint	Significado
Azul	Bounds del widget
Amarillo	Padding / margin
Verde	Overflow (→ <code>Wrap</code> , <code>SingleChildScrollView</code> , <code>maxLines+ellipsis</code>)
Rojo	Error de layout / constraints inválidas

- ✓ Widget no aparece: ¿existe en el tree? (si no, es bug de lógica)
- ✓ ¿`width: 0` / `height: 0` por constraints?
- ✓ ¿Overflow? → `Wrap` / `maxLines` + `TextOverflow.ellipsis`

Performance View: graba el timeline; cada frame debe completarse en **<16.67 ms (60fps)** / **<8.33 ms (120fps)**. Frame verde = OK, rojo = UI thread lento (build/layout), morado = raster thread lento (paint/compositing).

Threshold (Frame Analysis)	Si se supera
Widget rebuilds < 100 / frame	Usar <code>const</code> , extraer widgets
Layout / paint < 5 ms	Simplificar árbol, <code>RepaintBoundary</code>
Build > 8 ms en listas	<code>ListView.builder</code>
Shaders compilando cada frame	Impeller (default 3.10+); precache en web
Isolate bloqueado	<code>compute()</code> / isolate
Presión de GC	Menos alocações por frame

Diagnóstico: graba 5-10 segundos de interacción en `--profile` → identifica el frame más lento → flame chart → clasifica (build vs raster) → aplica el fix de la fila.

CPU Profiler: muestrea la CPU y genera flame chart. **Self time** = solo esa función; **Total time** = función + hijos. **Bottom Up** ordena por self time para encontrar las funciones calientes.

Señal CPU	Fix
Trabajo caro en <code>build()</code>	Hoist a <code>initState</code> con <code>late final</code>
<code>Image.network</code> re-descarga	<code>cacheWidth: 500</code> / <code>cached_network_image</code>
<code>TextPainter</code> domina	<code>maxLines</code> + <code>ellipsis</code> / <code>ConstrainedBox</code>
<code>AnimatedBuilder</code> rebuilda todo	Pasar un <code>child:</code> estable (se builda una vez)

Memory View: categorías **Dart Heap** (objetos Dart), **External** (nativo, imágenes), **GPU**. En los snapshots, ordena por **Retained Size** (objeto + todo lo que referencia) vs **Shallow Size** (objeto solo) vs **Instances**.

Operación	Uso
Take Heap Snapshot	Estado de la memoria en un momento
Diff Snapshot	Base → navegar a la pantalla 5-10× → comparar; clases que crecen = leak
Trace	Clase concreta: Allocations > Deallocations de forma estable = leak
GC	Colecciones de basura del isolate

Detectar leaks: snapshot base → navegar 5-10 veces a la pantalla sospechosa → diff snapshot → buscar `StreamSubscription` , `TextEditingController` , `Timer` , `AnimationController` que crecen → **fix:** cancelar/cerrar en `dispose()` .

6

Network, Logging y Debugger View

- **Network:** lista requests HTTP/WebSocket con body, headers y status. Verde 200-299, ámbar 300-399, rojo 400-599. Copia como cURL para probar fuera.
- **Timing de cada request:** DNS → TCP → TLS → Waiting → Response. DNS alto → cache DNS/IP; Waiting alto → backend/RLS/índices; Response alto → paginación/compresión.
- **Logging:** interpreta niveles de `package:logging` : FINE (verbose, gris) · INFO (azul) · WARNING (amarillo) · SEVERE (rojo) · SHOUT (fatal). Filtros por nivel mínimo, texto y tag.
- **Debugger View:** debugger web con el mismo VM Service: pausa, variables, call stack y **logpoints** sincronizados con VS Code.

```
// Sintaxis de filtros de Network
method:GET status:200 url:products
status:4xx # todos los client errors
-type:image # exclusión
method:POST status:200 # combinables

// Tags para filtrar Logging
debugPrint('[AUTH] ...') / '[API]' / '[BLOC]' / '[PRODUCTS]'
// Filtro: message:AUTH · Logging condicional: if (kDebugMode)
```

Recomendación: Network View para debugging de API (`debugPrint` del resultado + `Rows: ${data.length}`), Debug Console para lógica, Logging para trazas largas. `debugPrint` auto-throttle la salida en debug — prefírela a `print()` .

7 App Size: reducir el tamaño del build

ANALYZE-SIZE · UMBRALES · STORES

```
flutter build apk --analyze-size
flutter build appbundle --analyze-size
flutter build ios --analyze-size
# Reporte: build/app/outputs/flutter-size/snapshot-size.json
```

- **Google Play:** límite 200 MB para AAB. **App Store:** 2 GB binario; >200 MB no descargable por datos móviles.
- **deferred imports solo web** (Android/iOS compilan AOT).
- **Android:** `minifyEnabled true` + `shrinkResources true` + proguard.
- PNG→WebP, imágenes en red para no críticos, importar `widgets/painting` en vez de `material.dart` (tree-shaking); `flutter pub deps` para auditar peso.

Métrica	Saludable	Alerta
Total	< 15 MB	> 20 MB
Dart AOT	< 8 MB	> 10 MB
Assets	< 5 MB	> 8 MB
Imágenes	< 3 MB	> 5 MB

Pipeline: `--analyze-size` genera un reporte visual en Chrome DevTools App Size → identificar el paquete/asset dominante → optimizar (imágenes, `shrink`, `--split-debug-info`).

8 Rendimiento: rebuilds, keys y const

BUILD · PAINT · SELECTORS · 60FPS

El `build()` / `setState()` del padre rebuilda **todos los descendientes**. Objetivo: mantener build y raster < 8 ms (ideal < 4).

Técnica	Efecto
<code>const</code>	Reusa la misma instancia → no hay build del subtree (50-80% menos rebuilds en UIs estáticas)
<code>ListView.builder</code>	Solo construye items visibles
<code>BlocSelector</code> / <code>buildWhen</code>	Rebuilds selectivos del subtree según campo/condición
<code>RepaintBoundary</code>	Aísla el repintado (animaciones en listas)
<code>shouldRepaint</code>	<code>CustomPainter</code> solo repinta si cambia (<code>false</code> si estático)
Extraer widgets	Cada widget se reconstruye de forma independiente (<code>const ExpensiveChild</code>)

Keys

Key	Cuándo
<code>ValueKey(item.id)</code>	Listas dinámicas con id único — sin ella Flutter mezcla items al reordenar
<code>ObjectKey</code>	Identidad de objeto (objetos que no usan <code>==</code>)
<code>UniqueKey</code>	Forzar recreación completa del widget
<code>GlobalKey</code>	Acceder a estado desde fuera (forms, navegación)
Ninguna	Key sin motivo = overhead: solo úsala cuando Flutter necesita distinguir widgets

Anti-patterns de setState

- `setState` en loop → agrupa todas las mutaciones en **un** `setState`.
- `setState` en `didChangeDependencies` → puede reconstruir infinitamente; carga en `initState` + `async`.
- `setState` vacío sin cambio real.
- Mutar una colección: `_items.add(...)` no notifica → usa nueva referencia `_items = [..._items, item]`.

Regla: solo optimiza donde tengas un problema *medible* en profile mode. `BlocSelector` = un campo; `buildWhen` = varias condiciones. Agregar `RepaintBoundary` incorrectamente puede *empeorar* el rendimiento. Medir en debug no vale.

Patrones de leak (y su fix)

Leak	Fix
<code>StreamSubscription</code> sin cancelar	<code>_subscription?.cancel()</code> en <code>dispose()</code> (BLoC: en <code>close()</code>)
<code>TextEditingController</code> / <code>ScrollController</code>	<code>removeListener</code> antes de <code>dispose()</code>
<code>Timer.periodic</code> sin cancelar	<code>_timer.cancel()</code> en <code>dispose()</code>
Closure capturando <code>context</code>	Guard <code>if (mounted)</code> / flag <code>_isDisposed</code>
<code>setState</code> tras <code>await</code>	<code>if (!mounted) return</code> después de cada <code>await</code>
Listener no removido	<code>addObserver</code> / <code>removeObserver</code> simétricos

```
// Orden correcto en dispose()
_subscription?.cancel();
_animController.dispose();
_scrollController.removeListener(_onScroll);
_scrollController.dispose();
_textController.dispose();
super.dispose(); // siempre al final
```

Regla de oro: "Si creaste un controller o te suscribiste a un stream, DEBES cerrarlo en `dispose()`". Monitoriza con un `BlocObserver` (`onCreate/onClose/onError`) + diff snapshot para confirmar.

Rendering complejo

- >50 items → `ListView.builder` **obligatorio** (O(1) vs O(n) de `ListView`). Alto fijo → `SliverFixedExtentList` / `SliverPrototypeExtentList` .
- Imágenes: `cacheWidth` / `cacheHeight` = `(width * devicePixelRatio).toInt()` + `precacheImage` en `initState` .
- `Opacity` = "villano silencioso" (crea capa de composición): evítalo sobre subtrees grandes; usa `AnimatedOpacity` / `IgnorePointer` . `BackdropFilter` = coste muy alto. Prefiere `Decoration` con `borderRadius` sobre `ClipRRect` .

Memoria por imagen: 4000×3000 = 48 MB sin `cacheWidth` ; con redimensión a 200×150 ≈ 120 KB — la diferencia es ~**400×**.

Herramientas de diagnóstico que viven **en el código**, sin necesidad de abrir DevTools — ideales para validar `const`, rebuilds y repaints.

debugPrint vs print

- `print()` inunda el log de Android/iOS → riesgo de "Lost connection to device".
- `debugPrint()` acelera la salida y **no se ejecuta en release** (a diferencia de `print()`, que fuga info). **Regla de oro:** siempre `debugPrint()`.

Flags de render (de `package:flutter/rendering.dart`)

Flag	Qué muestra
<code>debugPaintSizeEnabled</code>	Bounds azules, padding amarillo, constraints punteadas
<code>debugPaintLayerBordersEnabled</code>	Bordes de capas GPU (flicker / zonas no repintadas)
<code>debugPaintPointersEnabled</code>	Regiones táctiles rojas
<code>debugPaintBaselinesEnabled</code>	Líneas base de texto
<code>debugPrintRebuildDirtyWidgets</code>	<code>Rebuilding: MyHomePage</code> — valida <code>const</code>
<code>debugPrintLayouts</code>	Log de cada paso de layout
<code>debugProfilePaintsEnabled</code>	Log de repaints — valida <code>RepaintBoundary</code>

Dump methods

Método	Cuándo
<code>debugDumpWidgetTree()</code>	Widget no visible / constraints inesperadas
<code>debugDumpRenderTree()</code>	Render objects, tamaño/posición exacta, overflow
<code>debugDumpLayerTree()</code>	Capas GPU: artefactos visuales / flicker
<code>debugDumpSemanticsTree()</code>	Árbol de accesibilidad (screen readers)

```
// Flags solo en debug (ignorados en profile/release)
WidgetsFlutterBinding.ensureInitialized();
assert(() {
  debugPaintSizeEnabled = true;
  debugPrintRebuildDirtyWidgets = true;
  return true;
})();

// Overlay de rendimiento en vivo
Stack(children: [MyApp(), PerformanceOverlay.all()]);

// Crash reporting + error UI custom
FlutterError.onError = (details) { FlutterError.presentError(details); /* Sentry en release */ };
ErrorWidget.builder = (details) => Material(child: Center(child: Text(...)));

// Invariantes de debug
assert(params.email.isNotEmpty, 'mensaje descriptivo');
assert(items.every((i) => i.quantity > 0), ...);
```

Aproximadamente el **80% de los bugs difíciles** de Flutter son asíncronos. Orden del event loop: síncrono → microtasks (`Future.microtask`) → timers (`Future.delayed`).

Bug	Fix
Future que nunca completa → loading infinito	<code>try/catch</code> en el <code>await</code> + <code>reset</code> en <code>catch</code>
<code>setState</code> tras desmontar → crash	<code>if (!mounted) return</code> tras cada <code>await</code>
Llamadas secuenciales → lenta	<code>Future.wait([fetchUsers(), fetchPosts()])</code>
Stream sin cancelar → leak	<code>_subscription.cancel()</code> en <code>dispose()</code>
<code>Completer</code> manual innecesario	<code>final result = await fetch();</code>
Trabajo pesado bloqueando el frame	<code>compute(_computeSum, 10_000_000)</code>

```
// Timeout SIEMPRE en red
fetch().timeout(10s, onTimeout: () => throw TimeoutException('...'));
posts.first.timeout(5s, onTimeout: () => throw TimeoutException('...'));

// Detectar streams silenciosos: wrapped en función con Stopwatch + logs
```

Patrón de carga segura: `try/catch` alrededor del `await` + estado `loading/error/data` + `timeout` + trabajo pesado delegado a `compute()` para no bloquear el UI thread.

12 Workflow por tipo de bug (26)

Tipo de bug	Herramienta principal	Vía rápida
UI se ve mal	Flutter Inspector → Select Widget + Debug Paint	Constraints, overflow → <code>Wrap</code> / <code>maxLines</code>
Estado inconsistente	Breakpoint en <code>emit()</code> + Bloc Inspector	<code>state.runtimeType</code> , verificar que <code>emit</code> corre, <code>mounted</code>
App lenta	Performance View (siempre <code>--profile</code>)	Frames >16 ms → flame chart → <code>const</code> /builder
API falla	Network View + cURL	Headers (Authorization), status, body; Supabase: RLS policies
Memory leak	Memory → diff snapshots	Clases que crecen → <code>dispose()</code> / <code>cancel()</code>
Build error	Terminal	Leer el error completo , tipos/imports/null-safety
Hot Reload no funciona	Hot Restart (<code>R</code>)	<code>initState()</code> / <code>main()</code> / <code>const</code> /imports no se recargan

Framework general: REPRODUCIR → AISLAR → DIAGNOSTICAR → CORREGIR → VERIFICAR (desarrollado en el §14 Playbook).

Debuggear es una **mentalidad, no una herramienta**. "No adivines, observa": la evidencia está en los datos, no en tu intuición ni en la IA. La IA acelera fixes solo cuando *ya entiendes* el bug.

Las 3 reglas	En la práctica
1. Nunca supongas, verifica	"Creo que es un null" → inspecciona la variable; "el estado no cambia" → breakpoint en <code>emit</code> ; "hay un rebuild innecesario" → <code>debugPrintRebuildDirtyWidgets</code>
2. Reduce el espacio de búsqueda	Binary search: divide el flujo por la mitad en cada paso (~ <code>log2(100)</code> = 7 iteraciones para 100 archivos)
3. Reproduce antes de arreglar	Si no puedes reproducirlo, no puedes arreglarlo

Usar IA	No usar IA ×
Sintaxis de un bug ya entendido	No sabes POR QUÉ falla
Boilerplate (p.ej. interceptor HTTP)	Lógica de negocio
Aprender patrones/paquetes	Un fix que solo "parece funcionar"
Explicar errores de compilación	Performance/memoria (la IA no puede medir)
	Entender el call stack completo (le falta el estado runtime)

Test de la dependencia: "¿Podrías explicarle a otro developer exactamente qué está pasando y por qué?" Si NO → aún no entiendes el bug. Y para pedir ayuda (humana o de IA), el checklist del §14 te dice si ya hiciste lo tuyo.

Un paso-a-paso reproducible para diagnosticar **CUALQUIER bug sin pedir ayuda a la IA**. Imprime esta sección y tenla al lado del monitor.

1 ENTENDER ¿Qué se esperaba vs qué pasó?	2 REPRODUCIR Steps exactos, siempre falla	3 AISLAR Binary search con breakpoints
4 DIAGNOSTICAR Datos, call stack, watch	5 CORREGIR Fix mínimo, un cambio a la vez	6 VERIFICAR Reproducir + tests + regresión

Paso 1 — ENTENDER

Responde **por escrito** las 7 preguntas (notepad, doc o comment):

- ✓ ¿Qué behavior esperaba ver?
- ✓ ¿Qué behavior veo en realidad?
- ✓ ¿Cuándo empezó a pasar? (¿antes funcionaba?)
- ✓ ¿Pasa siempre o solo a veces?
- ✓ ¿En qué dispositivo/plataforma?
- ✓ ¿Con qué datos de entrada?
- ✓ ¿Cambié algo recientemente? (`git log --oneline -10`)

Si no puedes responder la #1 → el bug no es técnico, es de requirements: habla con tu team/product owner antes de debuggear.

```
# Git como herramienta de debugging
git log --oneline -10
git diff HEAD~5 lib/presentation/bloc/login/login_bloc.dart
git stash && git checkout <hash> # probar si funcionaba antes
git checkout - && git stash pop # volver a HEAD y recuperar
```

Paso 2 — REPRODUCIR

Escribe los steps exactos con la acción OBSERVAR final:

```
1. Abrir la app → 2. Ir a login → 3. Escribir "test@email.com"
4. Escribir "12345678" → 5. Presionar "Login"
6. OBSERVAR: loading infinito
```

¿Cuándo falla?	Qué hacer
----------------	-----------

Con datos específicos	Probar con esos datos exactos
Después de cierto tiempo	Esperar ese tiempo
En cierto orden de acciones	Repetir el orden exacto
Con cierto estado previo	Forzar ese estado primero

Si no reproduces: logging detallado (`debugPrint`) en el flujo sospechoso · mismo entorno (dispositivo+OS) · reports de usuarios: ¿qué tienen en común? · DevTools → Logging en background.

Paso 3 — AISLAR (Binary Search Debugging)

En vez de leer 50 archivos, pon un breakpoint en la **mitad** del flujo: si el estado es correcto hasta ahí → el bug está después; si ya es incorrecto → está antes. En 3-4 iteraciones encuentras el paso exacto.

```
LoginSubmitted → validate → LoginUseCase → repository → api → emit(result)
Breakpoint 2 (validate): datos válidos ✓
Breakpoint 4 (repository): resultado Left(failure) ✗
→ el bug está entre el UseCase y el repository
```

Capa	Qué inspeccionar en el breakpoint
Evento	<code>event.email</code> , <code>event.password</code>
Validate	Resultado, errores
UseCase	<code>params</code> que recibe, <code>result</code> que retorna
Repository	Datos que envía, respuesta que recibe
API / Datasource	Request body, response, status code, headers
State	<code>state.runtimeType</code> , campos emitidos

Paso 4 — DIAGNOSTICAR

Herramientas en orden:

- 1. **VARIABLES** → ¿qué valor tiene la variable sospechosa?
- 2. **WATCH** → `state.runtimeType` , `result.runtimeType`
- 3. **CALL STACK** → ¿cómo llegué aquí?
- 4. **Debug Console** → probar "what if", `await`
- 5. `debugPrint()` → logging para el siguiente paso

```
// Expresiones para WATCH
state.runtimeType
result.runtimeType == Left<Failure, User>
myMap.keys.toList()
myList.length / myList.first / myList.isEmpty
(user as User).email
```

Paso 5 — CORREGIR

Regla	Tipo de bug	Fix típico
1 cambio a la vez · fix mínimo · sin features · commit tras el fix	Null safety	<code>?</code> o <code>?? default</code>
	Estado no se actualiza	Verificar que <code>emit()</code> se ejecuta
	Widget no se ve	Constraints, <code>Expanded</code> / <code>Flexible</code>
	API falla	Headers, URL, body
	Memory leak	<code>cancel()</code> en <code>dispose()</code>
	Performance	<code>const</code> , <code>RepaintBoundary</code> , <code>ListView.builder</code>

Paso 6 — VERIFICAR

- Reproducir los steps del Paso 2: ¿el bug ya no aparece?
- ¿Todos los tests existentes pasan?
- Probar el happy path completo + edge cases (vacío, límites)
- Probar en otro dispositivo/emulador (¿es específico de plataforma?)
- ¿Sin regresiones en funcionalidad relacionada?

Si algo se rompió: revierte el fix (`git checkout <archivo>`) → vuelve al Paso 3 → busca una solución que no rompa lo existente (o agrega un test que cubra el caso roto).

15 preguntas ANTES de pedir ayuda

- ☒ ¿Puedo reproducir el bug consistentemente?
- ☒ ¿Sé exactamente qué pasos lo provocan?
- ☒ ¿En qué archivo/función ocurre?
- ☒ ¿Qué valor tiene la variable sospechosa al fallar?
- ☒ ¿Qué dice el call stack?
- ☒ ¿Qué cambió recientemente? (git log)
- ☒ ¿Es de compilación o de runtime?
- ☒ ¿Es nuevo o existente?
- ☒ ¿Puse un breakpoint en el lugar correcto?
- ☒ ¿Inspeccioné con WATCH / Debug Console?
- ☒ ¿Leí el error COMPLETO?
- ☒ ¿Busqué en Google/Stack Overflow?
- ☒ ¿Revisé la documentación oficial del package/API?
- ☒ ¿Puedo explicárselo a otro developer?
- ☒ ¿Entiendo QUÉ falla (no solo CÓMO arreglarlo)?

Regla: >3 casillas sin marcar → estás pidiendo ayuda demasiado pronto. La respuesta está en los datos, no en tu intuición.

15

Debugging por capa: ejecución aislada con el debugger

MODO ANDAMIAJE SDD · SCRATCHPAD · REAL VS STUB

Cuando desarrollas una feature en **modo andamiaje SDD** (los cuerpos empiezan con `throw UnimplementedError()`), la feature no está cableada y no puedes verla desde la app completa. La técnica: crear un **scratchpad** (`tool/scratch/main_<capa>.dart`), correrlo con el debugger (F5) y verificar cada capa aislada a medida que la implementas — sin tests obligatorios ni dependencia de la IA.

Dos modos de verificación:

Modo	Cuándo	Qué verificas
A · Real (Integración)	Capas de datos (datasource → Supabase, repository → datasource)	Respuesta real de la API, excepciones reales, request en Network View
B · Aislado con stub	Capas superiores (usecase, cubit) cuando la dependencia inferior no está lista	Lógica de la capa con un stub manual (<code>Either<Failure,T></code> , transiciones de estado)

Capa	Runtime	Modo	Qué verificas
Entity / Model / State	<code>dart run</code>	—	Cálculos puros, roundtrip JSON, invariantes, Equatable/copyWith/props
UseCase	<code>dart run</code>	B stub	<code>Either<Failure,T></code> + mensajes exactos de validación
DataSource	<code>flutter run -t</code>	A real	Respuesta JSON real, AuthException/SocketException, RPC
RepositoryImpl	<code>flutter run -t</code>	A o B	Mapeo excepción → Failure, delega al datasource
Cubit	<code>flutter run -t</code>	B stub	Transiciones de estado (BlocObserver + Bloc Inspector)
Page/Widget	<code>flutter run</code>	A o B + Inspector	Render por cada estado (Loading/Loaded/Error)

Regla: inyecta dependencias a mano en el scratchpad, sin GetIt. Entra cada scratchpad en `launch.json` (agrúpalos con `presentation`). Secuencia: Entity → Model → DataSource → State → Repository → UseCase → Cubit → Page → Wiring. Verifica cada capa en cuanto la terminas. Capas puras → `dart run` ; capas con Flutter → `flutter run -t` .

Resultados reales de los escenarios (07, 18, 24)

- **Lista de 10.000 items** → `ListView.builder` : frame medio 125→12 ms, build 45→3 ms, memoria ~80→~15 MB.
- **Checkout con 4 bugs** → Inspector→ `Wrap` · Network→ `Future.wait` · Memory→ `dispose()` · CPU→ `RepaintBoundary` / `ValueListenableBuilder` .
- **API secuencial (perfil)** 3 llamadas 2900 ms → `Future.wait` → 1200 ms (-59%).
- **Chat Supabase con leak** → cancelar suscripción + `dispose()` : instancias 10→1, memoria +50→+2 MB.
- **Login parpadeante** → `BlocSelector` : 15→3 rebuilds/tecla, 12→1 ms.
- **Galería de imágenes** → `cacheWidth` : 12 MB→~120 KB por imagen.
- **Gráfica con `Timer.periodic`** → `ValueNotifier` + `ValueListenableBuilder` : buildScope 34%→8.5%.

Atajos VS Code que usarás a diario

Atajo	Acción	Atajo	Acción
F5	Start / Continue	Shift+F11	Step Out
Ctrl+F5	Run without Debugging	F9	Toggle breakpoint
Shift+F5	Stop	Ctrl+Shift+F9	Inline breakpoint
Ctrl+Shift+F5	Restart (hot restart)	Ctrl+Shift+Y	Debug Console
F10	Step Over	Ctrl+Shift+D	Run and Debug panel
F11	Step Into	r / R	Hot Reload / Hot Restart

Cheatsheets del módulo

Cheatsheet	Clave
06 (VS Code)	Atajos, tipos de breakpoint, error handling
17 (DevTools)	Vista por escenario + ideales de rendimiento
23 (Optimización)	Problema → causa → solución → DevTools View; mitos

Métricas saludables: build/raster < 4 ms (excelente) · > 8 ms = actuar · FPS ≥ 55 (ideal 60/120) · GC < 1-5/min · memory growth 0 MB/min (> 1 = leak) · heap idle < 50 MB · response < 500 ms · payload < 100 KB · árbol < 15 de profundidad · startup < 1 s. **Mitos:** `const` sí hace diferencia (50-80%) · `RepaintBoundary` mal usado puede empeorar · medir en debug no vale · keys sin motivo = overhead · slivers no siempre superan a `ListView.builder` .

Necesitas	Archivo
Modos debug/profile/release, componentes del debugger	01-fundamentos-debugging.md
launch.json, flutter attach, dart-define, args	02-configuracion-launch-json.md
Breakpoints conditional/function/logpoint, práctica con streams	03-breakpoints-avanzados.md
VARIABLES, WATCH, Debug Console como REPL	04-inspeccion-datos-consola.md
Multi-device, debugging remoto por red	05-multi-target-remoto.md
Atajos VS Code, hot reload/restart	06-cheatsheet-vscode.md
Escenarios BLoC/Supabase/GoRouter/UI + integrador	07-practicas-vscode.md
Cómo abrir DevTools, configuración	08-fundamentos-devtools.md
Widget tree, Select Widget, layout debugging	09-inspector-layout.md
Frames, flame chart, Enhance Tracing	10-performance-view.md
Sampling y flame chart de CPU	11-cpu-profiler.md
Heap, snapshots, GC, detección de leaks	12-memory-profiler.md
Requests HTTP, Supabase	13-network-view.md
Breakpoints y logpoints en DevTools	14-debugger-view.md
Niveles de package:logging, filtros	15-logging-view.md
analyze-size, límites de stores	16-app-size.md
Vista por escenario de debugging	17-cheatsheet-devtools.md
Ejercicios guiados + integrador	18-practicas-devtools.md
Modos, 60fps, herramientas de medición	19-fundamentos-rendimiento.md
const, builder, RepaintBoundary, BlocSelector	20-optimizar-rebuilds.md
Dispose, subscriptions, timers	21-memory-leak-detection.md
ListView.builder, Slivers, imágenes, CustomPainter	22-rendering-complejo.md
Tablas de diagnóstico, mitos, métricas	23-cheatsheet-optimizacion.md
6 escenarios reales + app "enferma"	24-practicas-optimizacion.md
5 bugs asíncronos, streams, isolates	25-debugging-asincrono.md
Workflow por tipo de bug + cheat sheet	26-workflow-debugging-por-tipo.md
Mentalidad "No adivines, observa", checklists, cuándo (no) usar IA	27-mentalidad-debugging.md
debugPrint, dump methods, debug flags, assertions	28-debugging-programatico-flutter.md
Playbook sistemático de 6 pasos, binary search, matriz de bugs	29-playbook-debugging-sistematico.md
Referencia rápida imprimible de TODO el módulo	30-referencia-rapida-flutter-debugging.md
Ejecución aislada por capa: scratchpad, real vs stub, matriz por capa	31-debugging-por-capa.md

Debugging Flutter en una línea: corre en debug para depurar, en profile para medir · pausa con breakpoints, condicionales y logpoints · inspecciona con Debug Console · diagnostica con DevTools (frames, heap, requests) · **no adivines, observa** — antes de pedir ayuda (humana o IA) responde tus 15 preguntas y sigue el playbook: entender → reproducir → aislar → diagnosticar → corregir → verificar · optimiza rebuilds y rendering solo donde haya un problema medible · cancela subscriptions en dispose · usa `debugPrint()` , flags y dump methods programáticos cuando no quieras abrir DevTools · y en modo andamiaje SDD, verifica cada capa aislada con un scratchpad y el debugger (`dart run` para lógica pura, `flutter run -t` para datos/estado/UI), real para la API y stub para lo que aún no existe.